

Evolutionary Contradiction Networks: Achieving Full Closure in Propositional Logic Discovery and the Emergence of a Solving-Learning Gap

Abhishek Srivastava
Independent Researcher

abhisri@gmail.com

May 2026

Abstract

*This paper identifies and characterizes the **closure-vs-learning gap**, a structural property of automated reasoning architectures that combine constructive solvers with experience-based learning. We demonstrate the gap empirically using ECN, a propositional logic discovery engine that achieves 100% closure on a structured benchmark of 735 solvable problems while its learning subsystem shows zero measurable transfer effect (M7=neutral across 755 matched pairs, eight orthogonal metrics). The gap was not designed or anticipated; it emerged purely through rigorous matched-pair measurement, revealing that the mechanism responsible for full closure operates entirely independently of the learning pipeline. The system solves everything but learns nothing from its synthesis successes, not because learning fails, but because the solver never fails in ways that make learning necessary. This suggests the gap may be a latent property of other hybrid solving-learning architectures that has gone undetected due to insufficient evaluation methodology. We formalize the tension and identify three architectural conditions (observation, abstraction, utility) required to bridge it.*

Keywords: automated reasoning, propositional logic, contradiction-driven evolution, learning gap, benchmark design, deterministic discovery

1. Introduction

Automated theorem proving and logic discovery systems have traditionally been evaluated on completeness: whether a system can, in principle, derive all valid conclusions from a given axiom set. Less attention has been paid to whether such systems *learn* from their successful derivations in ways that improve future performance. This paper presents a system that achieves the first property perfectly while revealing a fundamental tension with the second.

The Evolutionary Contradiction Network (ECN) is an automated discovery engine for propositional logic that accepts a goal formula and a set of axioms, then attempts to construct a derivation through an evolutionary process driven by contradiction detection and resolution. Unlike traditional resolution-based provers that operate in refutation space, ECN works constructively: it builds derivations forward from axioms toward the goal, using contradiction signals to guide its search.

Our evaluation uses a purpose-built benchmark derived from the LIMO methodology (Li et al., 2025), comprising 755 problem cells across five complexity tiers (L1–L5) and seven formula shape families (A–G),

including 20 expected-unsolvable cells that test the system's ability to correctly reject impossible goals. ECN achieves 100% solve rate on all 735 solvable cells and 100% correct rejection of all 20 unsolvable cells, with perfect cross-seed determinism ($M5=100\%$) and zero oracle leaks ($M6=0$).

However, the paper's central contribution is not the closure result itself but rather the identification of what we term the **closure-vs-learning gap**. ECN includes a technique library, a learning subsystem that stores and retrieves successful derivation patterns for reuse. Our Tier B evaluation protocol measures whether a pre-warmed library (containing techniques learned from auxiliary problems) improves solve outcomes on the benchmark problems. The result is unambiguously neutral: across 755 matched pairs comparing empty-library and staged-library execution, zero additional problems are solved by the staged configuration ($M7=\text{neutral}$, 0 wins, 0 regressions).

We trace this gap to the structural synthesis mechanism that enables closure. This mechanism operates by direct structural analysis of the goal formula, bypassing the learning pipeline entirely. The result is a system where the component responsible for 100% closure is architecturally disconnected from the component responsible for learning, producing perfect capability without transferable experience.

Crucially, this gap was not designed or anticipated. It emerged purely through measurement: our eight-metric evaluation framework detected it as an empirical property of the running system. This suggests it may be a latent feature of other hybrid solving-learning architectures that has gone undetected due to insufficient evaluation methodology. Any automated reasoning architecture that combines a powerful constructive solver with an experience-based learning mechanism faces the risk that the solver's capability renders the learning mechanism observationally irrelevant, not because learning is impossible, but because the solver never fails in ways that would make learned knowledge necessary.

2. Background and Related Work

2.1 Automated Reasoning in Propositional Logic

Classical automated theorem provers for propositional logic, from Davis-Putnam-Logemann-Loveland (DPLL) to modern CDCL solvers, operate primarily in the satisfiability domain. These systems determine whether a formula is satisfiable but do not construct derivations that demonstrate *how* a target formula follows from given axioms. Resolution-based methods (Robinson, 1965) and their extensions to multi-clause contradiction separation (Xu et al., 2025) work by refutation (assuming the negation of the goal and deriving contradiction) rather than by constructive forward derivation.

ECN occupies a different niche: it is a *constructive discovery engine* that builds derivation paths from axioms to goals using evolutionary search guided by contradiction signals. This positions it closer to program synthesis and inductive logic programming than to traditional SAT solving, though it operates entirely within propositional logic.

2.2 Learning in Automated Reasoning Systems

The integration of learning mechanisms into reasoning systems has a substantial history. Clause learning in CDCL solvers (Marques-Silva and Sakallah, 1999) demonstrated that storing conflict-derived clauses dramatically improves solver performance. More recently, neural-guided theorem provers (Bansal et al., 2019; Polu and Sutskever, 2020) use learned value functions to guide proof search. In reinforcement learning, experience replay (Mnih et al., 2015) provides a mechanism for reusing past successful episodes.

These approaches share an assumption: that learning from successful problem-solving episodes will improve future performance. Our work identifies a condition under which this assumption fails, not due to a deficiency in

the learning mechanism, but due to the structural properties of the solving mechanism that make learned knowledge unnecessary for achieving closure.

2.3 Neurosymbolic Architectures and Novelty Handling

Recent work in neurosymbolic cognitive architectures has explored systems that combine symbolic planning with learned components for handling novel situations. Goel et al. (2024) present a framework combining symbolic inference, neural perception, and novelty exploration modules, evaluated on procedural game environments with injected novelties. Their work demonstrates that combining symbolic and learned components can produce interference effects where the combined system underperforms individual components on certain metrics, a finding that resonates with the architectural tension we identify, though in a different domain and at a different level of analysis.

Other relevant work includes LIMO (Li et al., 2025), which demonstrates that small amounts of high-quality mathematical reasoning data can elicit sophisticated problem-solving in language models, and whose benchmark methodology inspired our evaluation framework.

3. System Architecture

ECN is implemented as a statically-typed module comprising approximately 4,200 lines of core logic. The system accepts a goal formula (propositional logic expression) and a set of axiom formulas, then searches for a constructive derivation path from axioms to goal.

3.1 Three-Tier Derivation Mechanism

ECN's derivation engine operates through three tiers, invoked in a fixed priority order:

Tier 1: Forward Chain. The system applies known derivation rules (modus ponens, conjunction introduction/elimination, disjunction rules, etc.) to the available formula set, expanding the set of derived formulas. This tier operates within a bounded combination budget (configurable; default MAX_COMBOS=64) and terminates when the goal is found in the derived set, the budget is exhausted, or no new derivations are possible.

Tier 2: Structural Synthesis. When forward chaining alone is insufficient, the system invokes a top-level synthesis mechanism that analyzes the goal formula's structure and determines whether the goal can be assembled from available derived components. This tier performs a feasibility check: can the goal, given its logical structure, be constructed from what forward chaining has produced? If feasible, it emits the construction directly.

Tier 3: Subgoal Extension. For goals that cannot be synthesized directly (because required subcomponents are not yet derived), the system decomposes the goal into subgoals and recursively attempts to derive each subgoal. This decomposition follows the goal's logical structure: a conjunction goal decomposes into its conjuncts, an implication goal generates a conditional subgoal, and so forth.

The three tiers form a complete derivation strategy for the propositional fragment: forward chaining handles all goals reachable within the combination budget, structural synthesis handles goals that require assembly beyond the budget but from available components, and subgoal extension handles goals requiring components that are not yet derived. Together, they achieve full closure on our benchmark.

3.2 Technique Library (Learning Subsystem)

ECN includes a technique library that records successful derivation strategies as reusable patterns. When the system successfully derives a goal, the derivation path is abstracted into a technique: a template capturing the sequence of rule applications, the structural relationship between inputs and output, and the conditions under which the technique is applicable.

The library supports export/import operations, allowing techniques learned in one session to be loaded into subsequent sessions. This mechanism is the basis of our Tier B evaluation: we warm the library on auxiliary problems, export the learned techniques, then measure whether importing them into a fresh session improves performance on the benchmark problems.

The technique library is architecturally positioned to receive output from all three derivation tiers. However, as we show in Section 6, the structural synthesis tier produces derivations that do not propagate into the library in a form that influences future solve attempts, creating the solving-learning gap.

3.3 Evolutionary Search and Contradiction Signals

The "evolutionary" aspect of ECN refers to its search strategy within the forward chaining tier. The system maintains a population of candidate derivation paths, scored by a combination of progress toward the goal (measured by structural similarity between derived formulas and goal subexpressions), contradiction reduction (formulas that resolve existing contradictions in the derivation set are preferred), and combination efficiency (shorter derivation paths are preferred over longer ones).

Contradiction signals serve as negative feedback: when a derivation path produces a contradiction (A and not-A both derived), the path is deprioritized and the contradiction is logged for potential use in indirect proofs. This evolutionary pressure drives the search toward consistent derivation paths that make genuine progress toward the goal.

4. Benchmark Design and Evaluation Methodology

4.1 Problem Curriculum

The evaluation benchmark comprises 755 problem cells organized along two dimensions: complexity level (L1–L5, with higher levels requiring deeper derivation chains and more complex intermediate constructions) and formula shape family (A–G, capturing different structural patterns in goal formulas). Of these 755 cells, 735 are expected-solvable and 20 are expected-unsolvable (belonging to a specific subfamily of Shape G that requires reasoning beyond propositional logic).

The benchmark methodology draws from LIMO (Li et al., 2025) in its emphasis on structured difficulty progression and evaluation of generalization, but targets propositional logic discovery rather than natural-language mathematical reasoning. Each problem cell is parameterized by a seed, enabling reproducible generation and cross-seed determinism testing.

4.2 Evaluation Metrics (M1–M8)

We define eight orthogonal metrics that collectively characterize system behavior:

Metric	Name	Definition
--------	------	------------

M1	Solve Rate	Fraction of solvable problems correctly solved + fraction of unsolvable problems correctly rejected
M2	Technique Reuse	Whether alpha-equivalent problems receive the same technique attribution
M3	Combination Efficiency	Mean number of rule-application combinations attempted before solution
M4	Fallback Usage	Count of random, shotgun, or maximum-likelihood-escape fallbacks invoked
M5	Cross-Seed Determinism	Whether the solve/no-solve outcome is identical across all seed values for each problem
M6	Oracle Leaks	Count of instances where the system uses the goal formula as an input (contamination)
M7	Cross-Mode Transfer	Difference in solve rate between staged-library and empty-library configurations
M8	Closure-Only Pass	Whether all solved problems are solved via legitimate closure (not fallback or oracle)

Table 1: The M1–M8 evaluation metric suite.

4.3 Tier B Protocol: Measuring Transfer

The Tier B evaluation protocol measures whether learned techniques transfer to improve solve outcomes. The protocol operates as follows: (1) For each of 5 seeds, a synthetic warmup curriculum of 20 auxiliary problems (structurally non-overlapping with the benchmark by construction) is solved, producing a library of 7 techniques per seed. (2) The library is exported as a snapshot. (3) The full 151-cell benchmark is run twice per seed: once with an empty library (Tier A mode) and once with the imported warmup library (staged mode). (4) Results are compared as matched pairs across 755 total pairs (5 seeds x 151 cells).

This protocol is designed to detect even marginal transfer effects. A single additional problem solved in staged mode (out of 755 pairs) would register as a positive M7 signal. The warmup curriculum uses W-prefix atoms and instance numbers above 9000 to guarantee non-overlap with benchmark atoms by construction, ensuring that any observed transfer must operate through structural generalization rather than literal pattern matching.

5. Results

5.1 Closure Performance

ECN achieves complete closure on the evaluation benchmark:

Metric	Value	Notes
M1 (solvable)	100% (1470/1470)	735 cells x 2 modes
M1 (unsolvable rejection)	100% (40/40)	20 cells x 2 modes
M2 (technique reuse)	100% (30/30 groups)	Per-mode alpha-family consistency
M3 (mean combos)	5.6 (Tier A) / 9.0 (Tier B)	After Class A pre-filter optimization
M4 (fallback usage)	0 / 0 / 0	Random / Shotgun / MLE in both modes
M5 (determinism)	100%	Across all 5 seeds
M6 (oracle leaks)	0	Across all 2,225 records
M7 (transfer)	Neutral (0 wins, 0 regressions)	755 matched pairs
M8 (closure-only)	100%	All solves via legitimate closure

Table 2: Complete benchmark results at the B-6 baseline.

5.2 Key Observations

Zero fallback reliance. The system never invokes random exploration, shotgun heuristics, or maximum-likelihood escape mechanisms ($M4=0$ across all modes). All solutions are produced through the three-tier deterministic mechanism.

Perfect determinism. Given the same problem specification, ECN produces identical solve/no-solve outcomes regardless of the random seed used for internal tie-breaking ($M5=100\%$). This is a stronger property than mere completeness; it indicates that the solution path is structurally determined by the problem, not by search luck.

No oracle contamination. The goal formula is never used as an axiom or derivation input ($M6=0$). Solutions are genuinely constructed from the provided axioms through legitimate inference rules. This is verified both by preflight manifest scanning and runtime defense-in-depth checks.

Efficient derivation. The mean combination count of 5.6 (Tier A) indicates that most problems are solved in very few inference steps, suggesting that the structural synthesis tier handles the majority of cases without extensive forward-chain search. The B-6 pre-filter optimization reduced mean combinations by approximately 95% from the pre-optimization baseline.

6. The Closure-vs-Learning Gap

6.1 Characterizing the Gap

The central finding of this paper is a structural tension between ECN's closure capability and its learning capability. We define the **closure-vs-learning gap** as follows:

A system exhibits a closure-vs-learning gap when its mechanism for achieving complete problem coverage operates independently of its mechanism for accumulating transferable experience, such that full closure is achievable without the learning mechanism contributing to or benefiting from the closure-producing derivations.

In ECN, this manifests concretely: the structural synthesis tier (Tier 2) achieves 100% solve rate on all benchmark problems (M1=100%), while the technique library shows zero transfer effect (M7=neutral). The library is populated (after warmup, it contains 7 techniques per seed), but these techniques are never the difference between solving and not solving a benchmark problem.

6.2 Architectural Analysis

The gap arises from a specific architectural property. The structural synthesis mechanism operates by analyzing the goal formula's AST (abstract syntax tree) and determining whether each subexpression can be constructed from the available derived formula set. This analysis is purely structural: it depends on what formulas are currently available and what the goal requires, with no reference to previously learned patterns.

Critically, this mechanism is *sufficient* for closure. Every problem in the benchmark that forward chaining alone cannot solve is handled by structural synthesis or subgoal extension. The technique library, which stores patterns learned from prior solutions, is architecturally available but observationally unnecessary: there is no problem that the library solves that synthesis cannot also solve, and synthesis does not consult the library.

This creates a precise characterization of the gap: the system has both the ability to solve (via synthesis) and the ability to learn (via the technique library), but these abilities operate on parallel tracks that do not intersect at the point of solving. Learning happens, but it never matters for closure.

6.3 Formal Properties of the Gap

We can state the gap's properties precisely using the metric suite:

- **M1 independence from M7:** M1=100% regardless of whether M7 is positive, neutral, or negative. Closure does not depend on transfer.
- **M7 neutrality despite library population:** The staged library contains 7 techniques per seed (non-trivial; warmup succeeds), yet contributes 0 additional solves. The library is populated but inert with respect to benchmark problems.
- **M4=0 confirms synthesis dominance:** Zero fallback invocations means the system never enters a state where learned techniques *could* rescue a failed attempt. Synthesis prevents the failure conditions that would make learning visible.
- **M3 asymmetry hints at interference:** Mean combinations are 5.6 in empty mode vs. 9.0 in staged mode (+60%). The library introduces additional candidate evaluations that slightly increase work without changing outcomes. This is not harmful (M1 is identical) but suggests library presence adds overhead without benefit.

6.4 The Gap Is Not a Bug

It is important to distinguish the closure-vs-learning gap from a system failure. The gap is a *structural property* of the architecture, not a deficiency in either component. The synthesis mechanism is correctly achieving its design goal (full closure). The learning mechanism is correctly storing and retrieving techniques. The gap emerges from their relationship: synthesis is powerful enough to make learning unnecessary for the closure objective.

This observation has a specific implication for system development: bridging the gap requires neither fixing the solver (it works perfectly) nor fixing the learner (it stores correctly), but rather creating an architectural pathway through which synthesis successes can inform the learning pipeline in ways that produce observable transfer effects. The challenge is not capability but connectivity.

7. Discussion

7.1 Implications for Hybrid Reasoning Architectures

The closure-vs-learning gap raises a general question for automated reasoning systems that combine constructive solvers with learning mechanisms: under what conditions does a powerful solver render its companion learning mechanism observationally inert?

We conjecture that this pattern is more common than recognized. In any system where one component achieves ceiling performance on the evaluation set, companion components that operate below that ceiling will appear to contribute nothing, even if they would become critical under harder conditions or different evaluation criteria. The gap is not merely about the learning mechanism being weak; it is about the evaluation conditions being insufficient to expose the solver's limits.

This suggests a methodological principle: *when evaluating hybrid architectures, design benchmark conditions that specifically expose the limits of each component in isolation*. If a single component achieves perfect performance, the benchmark cannot distinguish between "the other components are useless" and "the other components would help under harder conditions."

7.2 Conditions for Bridging the Gap

Based on our analysis, bridging the closure-vs-learning gap requires creating conditions under which synthesis-produced derivations are *observable* by the learning pipeline and *representable* in the technique library's abstraction format. Specifically, three architectural conditions must be met:

(1) **Observation:** The learning mechanism must be able to observe derivation events produced by the synthesis tier, not just the forward-chain tier. Currently, synthesis operates as a closed subsystem whose internal steps are not exposed to the technique abstraction pipeline.

(2) **Abstraction:** Synthesis-produced derivations must be representable in a form that the technique library can store and later match against new problems. This requires a shared representational vocabulary between synthesis output and library storage.

(3) **Utility:** Stored synthesis-derived techniques must be applicable to future problems in a way that provides an advantage over re-running synthesis from scratch. This is the most challenging condition: if synthesis is fast and reliable, why would retrieving a stored pattern be preferable?

The answer to condition (3) likely involves efficiency rather than capability: a stored technique could allow the system to bypass the synthesis analysis entirely, solving a problem in one step (pattern match + instantiate) rather than multiple steps (forward chain + synthesis analysis + construction). Under harder problems or tighter

resource budgets, this efficiency gain could become the difference between solving and timing out.

7.3 Limitations

Several limitations bound our findings. First, the benchmark operates entirely within propositional logic; the closure-vs-learning gap may not manifest (or may manifest differently) in first-order or higher-order settings where structural synthesis is necessarily incomplete. Second, our benchmark's difficulty range (L1–L5) may not be sufficient to expose conditions under which the learning mechanism becomes necessary; harder curricula (which we term Tier C and beyond) may produce different M7 results. Third, the system's perfect determinism (M5=100%) means our results may not generalize to systems with stochastic search components.

Additionally, the technique library's current abstraction format may be too coarse to capture the structural insights that synthesis produces. A richer technique representation could potentially bridge the gap without architectural changes to the synthesis mechanism itself. We have identified specific design candidates for this richer representation but their evaluation is deferred to future work.

8. Conclusion and Future Work

We have presented ECN, an automated discovery engine that achieves 100% closure on a structured propositional logic benchmark of 735 solvable problems plus 20 unsolvable controls, with perfect determinism, zero oracle leaks, and zero fallback reliance. More significantly, we have identified and characterized the closure-vs-learning gap: a structural property of hybrid solving-learning architectures in which a sufficiently powerful solver renders its companion learning mechanism observationally inert.

This finding contributes to the understanding of automated reasoning system design in two ways. First, it demonstrates that full closure and effective learning are not automatically co-produced: a system can achieve one perfectly while the other shows no measurable effect. Second, it identifies specific architectural conditions (observation, abstraction, utility) that must be met for the gap to be bridged.

Future work will focus on three directions: (1) extending the benchmark to harder curricula (Tier C) that may expose the solver's limits and create conditions where learning becomes necessary; (2) implementing the architectural bridge between synthesis and learning through richer technique representations; and (3) evaluating whether the gap pattern appears in other domains where constructive solvers coexist with experience-based learning systems.

The code, benchmark specification, and full evaluation data are available under CC BY-NC-SA 4.0 license. A preliminary (incomplete) version was registered at OSF (osf.io/c2va4) on June 24, 2025.

References

- [1] Bansal, K., Loos, S., Rabe, M., Szegedy, C., and Wilcox, S. (2019). HOList: An environment for machine learning of higher order logic theorem proving. *Proceedings of ICML 2019*.
- [2] Davis, M., Logemann, G., and Loveland, D. (1962). A machine program for theorem-proving. *Communications of the ACM*, 5(7):394–397.
- [3] Goel, S., Shrestha, R., Md, A.K., Murali, K., Vattam, S., and Goel, A. (2024). A neurosymbolic cognitive architecture framework for handling novelties in open worlds. *Artificial Intelligence*, 331:104111.
- [4] Li, C., Huang, W., Tang, Y., et al. (2025). LIMO: Less is more for reasoning. *arXiv preprint arXiv:2502.03387*.
- [5] Marques-Silva, J.P. and Sakallah, K.A. (1999). GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5):506–521.

- [6] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533.
- [7] Polu, S. and Sutskever, I. (2020). Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*.
- [8] Robinson, J.A. (1965). A machine-oriented logic based on the resolution principle. *Journal of the ACM*, 12(1):23–41.
- [9] Xu, Y., Liu, J., Zhong, L., and Chen, S. (2025). Dynamic automated deduction by contradiction separation: The standard extension algorithm. *arXiv preprint arXiv:2510.08468*.